

CS3002E Compiler Design

Monsoon 2025

Lecture #17

Bottom-Up Parsing - 6: LR(1) Parsing

Department of Computer Science & Engineering
NIT Calicut

Review

- LR(0) Parser
- SLR Parser

Exercise

Check if the following grammars is (a) LR(0) (b) SLR?

$$S \rightarrow L = R \mid R$$

$$L \rightarrow *R \mid id$$

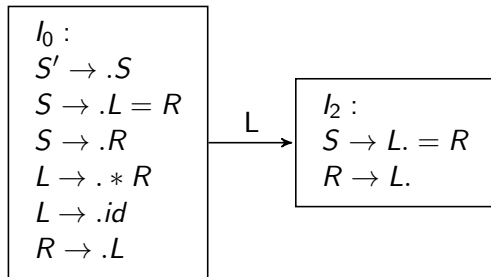
$$R \rightarrow L$$

$S' \rightarrow S$

$S \rightarrow L = R \mid R$

$L \rightarrow *R \mid id$

$R \rightarrow L$



$$S' \rightarrow S$$

$$S \rightarrow L = R \mid R$$

$$L \rightarrow *R \mid id$$

$$R \rightarrow L$$

$I_2 :$

$$S \rightarrow L. = R$$

$$R \rightarrow L.$$

- shift/reduce conflict in state 2, not LR(0).
- Is it SLR?
- *FOLLOW*(*R*)?

Exercise

$$S' \rightarrow S$$

$$S \rightarrow L = R \mid R$$

$$L \rightarrow *R \mid id$$

$$R \rightarrow L$$

Do rightmost derivation of $*id = id$

$$S' \rightarrow S$$

$$S \rightarrow L = R \mid R$$

$$L \rightarrow *R \mid id$$

$$R \rightarrow L$$

$I_2 :$

$$S \rightarrow L. = R$$

$$R \rightarrow L.$$

Not SLR. shift/reduce conflict in state 2 for lookahead =

- shift =
- reduce by $R \rightarrow L$ ($= \in FOLLOW(R)$)

Example: check if grammar is SLR

$$S' \rightarrow S$$

$$S \rightarrow L = L \mid id$$

$$L \rightarrow id$$

Example

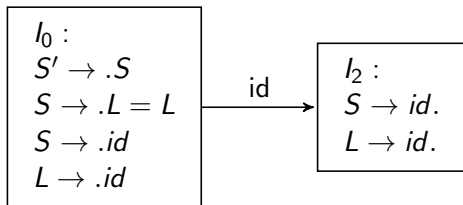
$$S' \rightarrow S$$

$$S \rightarrow L = L \mid id$$

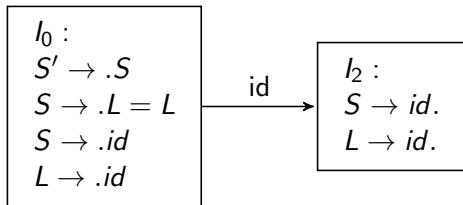
$$L \rightarrow id$$

Grammar is not SLR

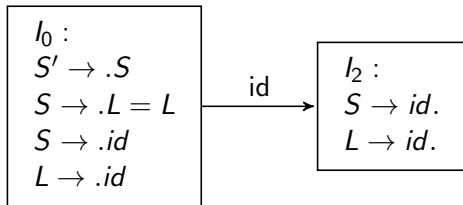
- *reduce/reduce* conflict in state $\{S \rightarrow id., L \rightarrow id.\}$
\$ is in both FOLLOW(S) and FOLLOW(L).



reduce/reduce conflict in $ACTION[2, \$]$



- parser reaches state 2 after reading an id .
- in state 2, reduction by
 - $S \rightarrow id$ is valid only if lookahead is $\$$
 - $L \rightarrow id$ is valid only if lookahead is $=$



- in state 2, reduction by $S \rightarrow id$ is valid only if lookahead is \$ and reduction by $L \rightarrow id$ is valid only if lookahead is =
- the required additional information can be kept as a second component of an item.

LR(1) Items

- An LR(1) item has two components
 - first component is similar to LR(0) item
 - the second component is the **lookahead** of the item.
- Items of the form $[A \rightarrow \alpha.\beta, a]$
 - a is a lookahead symbol (a is either a terminal or the right endmarker \$)
- $[A \rightarrow \alpha., a]$ in a state indicates that the symbol a can follow handle α for which there is a possible reduction to A .
 - reduction by $A \rightarrow \alpha$ only if lookahead is a^1

¹set of such a 's will be a subset of FOLLOW(A)

LR(1) Item

$$S' \rightarrow S$$

$$S \rightarrow L = L \mid id$$

$$L \rightarrow id$$

LR(0) items in state 2: $\{S \rightarrow id., L \rightarrow id.\}$

reduction by $L \rightarrow id$ if lookahead is =

indicated by LR(1) item $[L \rightarrow id., =]$

LR(1) items in state 2 ?

$$\begin{aligned}
 S' &\rightarrow S \\
 S &\rightarrow L = L \mid id \\
 L &\rightarrow id
 \end{aligned}$$

State 2: $\{S \rightarrow id., L \rightarrow id.\}$

- reduction by $S \rightarrow id$ if lookahead is $\$$
- reduction by $L \rightarrow id$ if lookahead is $=$

Set of LR(1) items $\{[S \rightarrow id., \$], [L \rightarrow id., =]\}$

- each item indicates the lookahead for which the particular reduction is valid.

LR(1) Parsing

LR Parsing with one symbol lookahead

- more powerful than SLR parser
- also known as *Canonical LR* or just *LR*
- uses LR(1) items

LR(1) Parser - Initial state

Initial State is represented by the set of items in
 $Closure(\{[S' \rightarrow .S, \$]\})$

Closure of Set of LR(1) Items

For a set of items I , $Closure(I)$ is computed as follows:

- Add every item in I to $Closure(I)$.
- If $[A \rightarrow \alpha.B\beta, a]$ is in $Closure(I)$, then for each production $B \rightarrow \gamma$ and for each $b \in FIRST(\beta a)$, add $[B \rightarrow .\gamma, b]$ to $Closure(I)$.
- Repeat the above step until no more new items can be added to $Closure(I)$.

Example

$$S' \rightarrow S$$

$$S \rightarrow L = L \mid id$$

$$L \rightarrow id$$

Initial State, I_0 : $Closure(\{[S' \rightarrow .S, \$]\})$

$$= \{[S' \rightarrow .S, \$],$$

$$[S \rightarrow .L = L, \$],$$

$$[S \rightarrow .id, \$],$$

$$[L \rightarrow .id, =]\}$$

Example

$$S' \rightarrow S$$

$$S \rightarrow L = L \mid id$$

$$L \rightarrow id$$

$$\begin{aligned} \text{Initial State, } I_0: \text{Closure}(\{[S' \rightarrow .S, \$]\}) \\ = \{[S' \rightarrow .S, \$], \\ [S \rightarrow .L = L, \$], \\ [S \rightarrow .id, \$], \\ [L \rightarrow .id, =]\} \end{aligned}$$

Note the lookahead $=$ in the LR(1) item $[L \rightarrow .id, =]$

$=$ is the symbol that the parser can expect after getting a string derivable from L .

lookahead component $=$ is obtained by computing
 $FIRST(=L\$)$

LR(1) Parser - Goto transitions

$Goto(I, X)$ for state I is computed as follows:

For each item in I of the form $[A \rightarrow \alpha.X\beta, a]$, add $Closure(\{[A \rightarrow \alpha X.\beta, a]\})$ to $Goto(I, X)$

Example

$$S' \rightarrow S$$

$$S \rightarrow L = L \mid id$$

$$L \rightarrow id$$

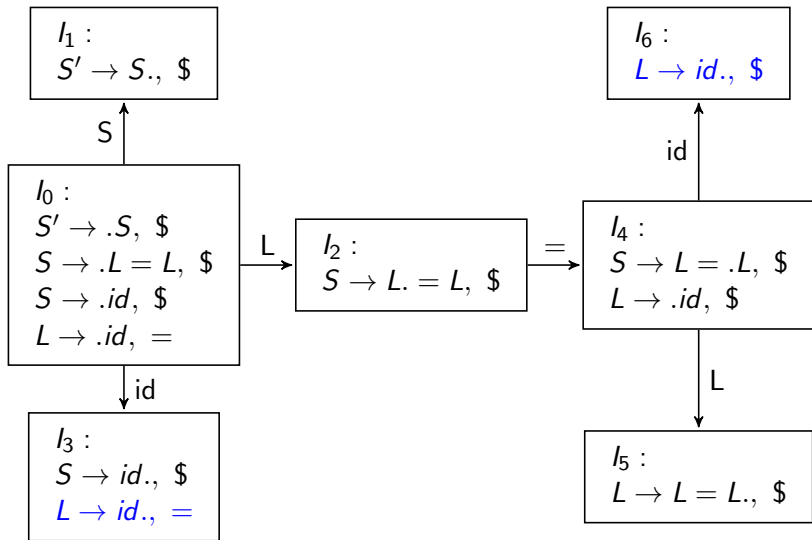
$$\begin{aligned} \text{Initial State, } I_0: & \text{Closure}(\{[S' \rightarrow .S, \$]\}) \\ &= \{[S' \rightarrow .S, \$], \\ & \quad [S \rightarrow .L = L, \$], \\ & \quad [S \rightarrow .id, \$], \\ & \quad [L \rightarrow .id, =]\} \end{aligned}$$

$$\text{Goto}(I_0, S) = \{[S' \rightarrow S., \$]\}$$

$$\text{Goto}(I_0, L) = \{[S \rightarrow L. = L, \$]\}$$

$$\text{Goto}(I_0, id) = \{[S \rightarrow id., \$], [L \rightarrow id., =]\}$$

0 : $S' \rightarrow S$ 1 : $S \rightarrow L = L$ 2 : $S \rightarrow id$ 3 : $L \rightarrow id$



LR(1) Parsing Table

Let $C = I_0, I_1, \dots, I_n$, be the collection of sets of LR(1) items. For each set I_i , the parser has a corresponding state i . The parsing table has two parts - ACTION and GOTO. The entries for state i are as follows:

- If $[A \rightarrow \alpha.a\beta, b]$ is in I_i , and $Goto(I_i, a) = I_j$, set $ACTION[i, a]$ to s_j (shift and goto state j)
- If $[A \rightarrow \alpha., a]$ is in I_i , $A \neq S'$, set $ACTION[i, a]$ to r_j (reduce using production numbered j , assuming $A \rightarrow \alpha$ is numbered j)
- If $[S' \rightarrow S., \$]$ is in I_i , set $ACTION[i, \$]$ to *accept*
- If $Goto(I_i, A) = I_j$ then set $GOTO(i, A)$ to j
- All undefined entries are made *error* (blank entries)

LR(1) Parsing Table

$0: S' \rightarrow S$ $1: S \rightarrow L = L$ $2: S \rightarrow id$ $3: L \rightarrow id$

state	id	=	\$	S	L
0	s3			1	2
1			acc		
2		s4			
3		r3	r2		
4	s6				5
5			r1		
6			r3		

Exercise 1

Check if the following grammar is LR(1)? If YES, build LR(1) parsing table for the grammar.

$$S \rightarrow AA$$

$$A \rightarrow aA \mid b$$

Exercise 2

Check if the following grammar is LR(1)? If YES, build LR(1) parsing table for the grammar.

$$S \rightarrow L = R \mid R$$

$$L \rightarrow *R \mid id$$

$$R \rightarrow L$$

References:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU chapter 4, section 4.7